

The Real Problem

- Training is a batch job.
- Inference is a latency-sensitive, concurrent, stateful service.
- Most courses teach training.
- Almost none teach production serving.
- This talk is about that second half.

Training and Inference Are Not Symmetric

Training

- Batch job — runs once
- Optimize for throughput, not latency
- Single process is usually fine
- Failure = re-run the job
- Hardware: maximize GPU utilization
- Metric: loss / accuracy



Inference

- Live service — runs forever
- Optimize for P99 latency under load
- Must handle concurrent requests
- Failure = lost revenue or user harm
- Hardware: efficiency + availability
- Metric: latency · throughput · cost

Three Distinct Problems — Never Conflate Them

01 Throughput



Serving N requests/sec when the model takes T ms each

⚠ GIL, memory, GPU contention → breaks ~100 rps with naive Python

02 Latency



P99 latency must stay under SLA even under burst traffic

⚠ Queue saturation destroys tail latency — P99 diverges from P50

03 Resource Efficiency



GPU utilization must stay high — idle GPUs are expensive

⚠ 1 model per GPU → 10–30% utilization. Rest is waste.

Why Python + Naive Serving Breaks

The GIL Problem

Python's Global Interpreter Lock means a single process can execute only one thread at a time.

For CPU-bound work like model inference, this is catastrophic.

The fix: multiple processes — not threads. Ray's actor model is the idiomatic answer.

The GPU Memory Problem

Loading a 7B parameter model:

FP32: 14 GB GPU memory

BF16: 7 GB GPU memory

4 naive replicas = 56 GB

The correct approach: load once, serve from multiple workers sharing that context — a fundamentally different architecture.

**Ray is a distributed execution
framework that lets you scale
Python across cores, machines,
and GPUs —**

*without rewriting your code
into a distributed system.*



One Runtime. Five High-Level Libraries.



Ray Core

Distributed runtime —
Tasks & Actors. The
substrate everything else
builds on.



Ray Data

Distributed data
preprocessing at scale.
Streaming + batch.



Ray Train

Distributed training
wrappers — PyTorch,
TensorFlow, XGBoost.



Ray Tune

Hyperparameter search
at scale across a cluster.



Ray Serve

Production model
serving. The focus of this
talk.

TODAY

One Decorator. Infinite Scale.

```
import ray
ray.init()

# A regular Python function
def add(a, b):
    return a + b

# A Ray remote function — runs on a worker process
@ray.remote
def add_remote(a, b):
    return a + b

# Non-blocking — returns a future immediately
future = add_remote.remote(1, 2)

# Block and retrieve
result = ray.get(future)    # result = 3
```

@ray.remote

Serializes the function, ships it to a worker process, executes it.

.remote()

Returns a future immediately — the caller never blocks.

ray.get()

Block and retrieve one or many futures at once.

Object Store

Result goes into shared in-memory object store — accessible cluster-wide.

Tasks vs Actors — Stateless vs Stateful

```
# TASK — stateless, pure function
@ray.remote(num_cpus=1, num_gpus=0.5)
def preprocess_batch(batch):
    return [tokenize(t) for t in batch]

# 100 concurrent jobs, each gets 0.5 GPU
futures = [preprocess_batch.remote(b)
            for b in batches]
results = ray.get(futures)
```

Tasks = preprocessing, feature engineering, any pure function.

```
# ACTOR — stateful, long-lived
@ray.remote(num_gpus=1)
class ModelActor:
    def __init__(self, model_path):
        # Runs ONCE — model loaded here
        self.model = load_model(model_path)

    def predict(self, inputs):
        # Called many times — model stays
        return self.model(inputs).tolist()

actor = ModelActor.remote('resnet50.pt')
future = actor.predict.remote(batch)
```


Shared Memory — Zero Copy Within a Node

```
# Put 1GB embedding matrix into object store ONCE
large_tensor_ref = ray.put(embedding_matrix)

# Multiple workers access it — zero copy on same node
@ray.remote
def compute_similarity(query_vec, embeddings_ref):
    embeddings = ray.get(embeddings_ref) # mmap, no copy
    return cosine_similarity(query_vec, embeddings)

# All 100 workers share the same physical memory
futures = [compute_similarity.remote(q, large_tensor_ref)
            for q in queries]
```

Cross-node: transferred once over network, cached locally on worker node.

~10 GB/s

Same-node access
(shared memory)

**Apache
Plasma**

Backend store
(backed by /dev/shm)

**Apache
Arrow**

Serialization for
numpy/pandas — zero copy

pickle

Fallback for
arbitrary Python objects

Ray Serve

Production-Grade Inference Engine

- HTTP Ingress (ASGI/Starlette)
- Deployment Graph — model DAG
- Autoscaling on queue depth
- Request batching — adaptive
- Traffic splitting / canary
- Model multiplexing

What it replaces:

Flask/FastAPI wrapper

No process management, no batching, no scaling

Nginx + Gunicorn

No model-aware routing or queue-depth autoscaling

Kubernetes alone

K8s doesn't manage GPU assignment or actor lifecycle

Triton (alone)

Python-native logic, multi-model graphs need more than Triton

KubeRay: Ray Serve on Kubernetes — the production standard.

A Production Deployment — 40 Lines

```
from ray import serve
from transformers import pipeline

@serve.deployment(
    num_replicas=4,                # 4 parallel replicas
    ray_actor_options={ 'num_gpus': 1 }, # each gets 1 GPU
    autoscaling_config={
        'min_replicas': 2,
        'max_replicas': 10,
        'target_num_ongoing_requests_per_replica': 5
    }
)
class TextClassifier:
    def __init__(self):
        self.model = pipeline(
            'text-classification',
            model='distilbert-base-uncased',
            device=0
        )

    async def __call__(self, request):
        data = await request.json()
        return { 'predictions': self.model(data['texts']) }

app = TextClassifier.bind()
serve.run(app, route_prefix='/classify')
```

num_replicas=4

Four processes, four GPUs, fully concurrent

autoscaling_config

Scales on queue depth — semantically correct for inference

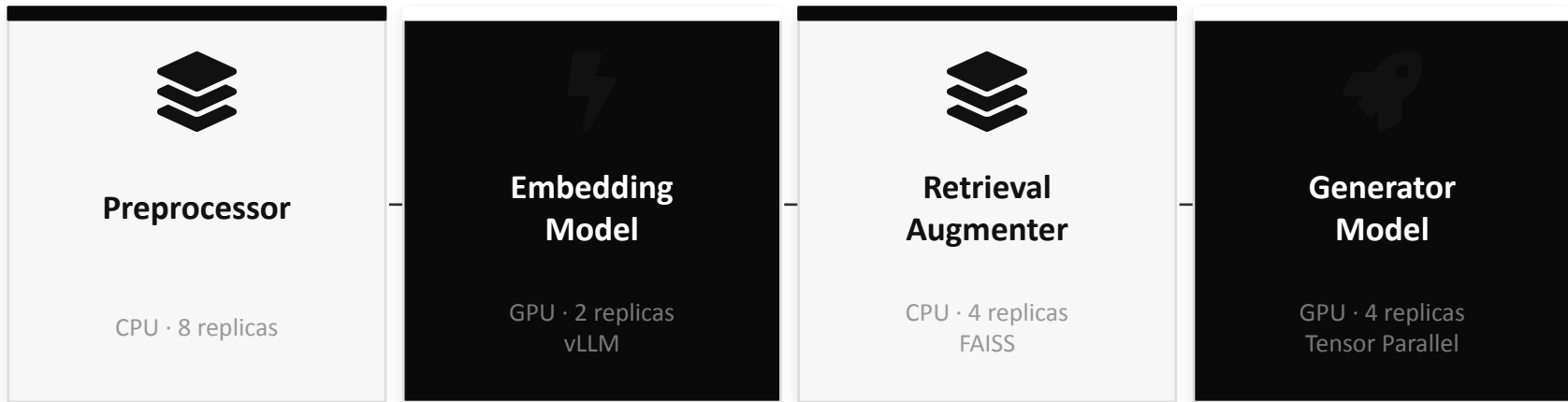
async __call__

Async handler — non-blocking I/O between requests

serve.run()

One call — HTTP endpoint live on the cluster

Deployment Graph — Models Scale Independently



HTTP Request → Preprocessor → Embedding → Retrieval → Generator → HTTP Response

Each stage: independent autoscaling · independent failure isolation · independent resource allocation

```
pipeline = RetrievalAugmenter.bind(EmbeddingModel.bind())
serve.run(pipeline)    # Wire the graph — one line
```

Scale on Queue Depth — Not CPU Utilization

Traditional K8s autoscaler:

Scale on CPU % — wrong for GPU inference.
CPU is cheap. GPU is the bottleneck.

Ray Serve autoscaler:

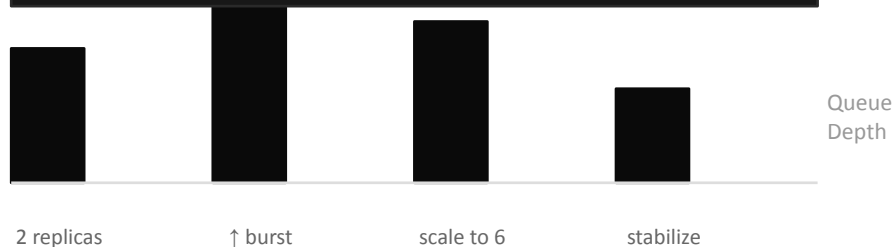
Scale on **pending request queue depth per replica**.

When a replica has more than
target_num_ongoing_requests_per_replica requests
queued → spin up a new replica.

This is semantically correct for inference.

Queue depth triggers scale-up

```
autoscaling_config={
    'min_replicas': 2,
    'max_replicas': 10,
    # Scale up when queue > 5 requests/replica
    'target_num_ongoing_requests_per_replica':
5
}
```



Continuous Batching — How to Max Out GPU Throughput

Static Batching

Naive approach

Collect 32 requests. Run inference. Return.

Problem: request 33 waits for the entire 200ms batch to complete.

Result: P99 explodes under load.

Continuous Batching

ORCA (2022) · vLLM · TGI

New requests join at each forward pass dynamically.

Completed sequences evict. Batch size varies 1 → max_batch_size.

Result: Throughput stays high. Tail latency bounded.

```
@serve.batch(max_batch_size=32, batch_wait_timeout_s=0.05)
async def batched_predict(self, inputs: list[str]) -> list[float]:
    # Called when batch is full OR 50ms timeout expires
    return self.model.predict_batch(inputs) # up to 32 items
```

Model Multiplexing — Many Models, One GPU Pool

Multi-tenant scenario: 50 fine-tuned models, one GPU pool. LRU cache — load on demand, evict on miss.

```
from ray.serve.handle import DeploymentHandle

@serve.deployment(ray_actor_options={'num_gpus': 1})
@serve.multiplexed(max_num_models_per_replica=5) # LRU: keep
5 models warm
class MultiplexedClassifier:

    async def load_model(self, model_id: str):
        # Called automatically by Ray Serve on cache miss
        return load_model_from_registry(model_id)

    async def __call__(self, request):
        model_id = request.headers.get('X-Model-ID')
        model = await self.get_multiplexed_model(model_id)
        data = await request.json()
        return model.predict(data['text'])
```

LRU Cache

`max_num_models_per_replica` controls cache size

Cold Start

Model loads on first request for that ID — warm after

Routing

X-Model-ID header routes to correct model

Economics

50 models on 10 GPUs vs 50 GPUs — 5x cost reduction

The Production LLM Stack

KubeRay

Kubernetes operator — cluster lifecycle

Ray Serve

HTTP, autoscaling, deployment graph, traffic splitting

vLLM

PagedAttention — eliminates KV cache fragmentation · 3-5x throughput vs naive HF

Ray (tensor parallel)

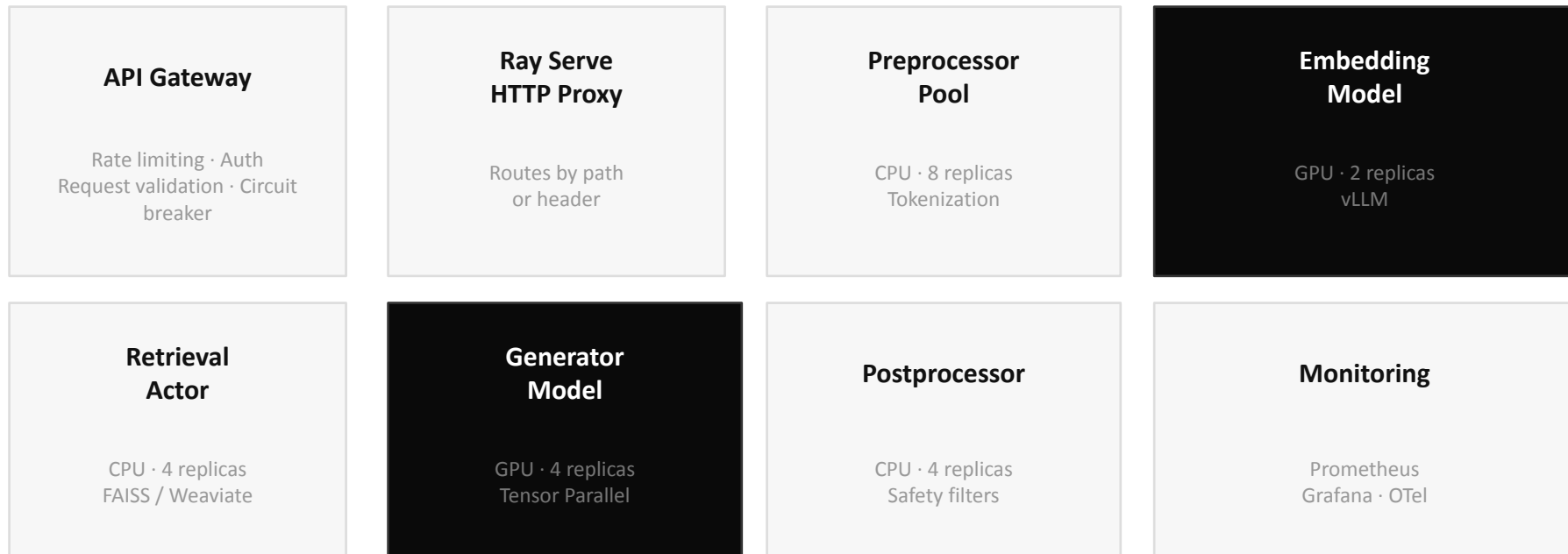
Model sharded across N GPUs — each GPU an actor. 70B+ models in production.

Model Weights

BF16 / INT4 (AWQ/GPTQ) — quantized for memory efficiency

vLLM throughput: 3-5x better than naive HuggingFace · **PagedAttention** eliminates KV cache fragmentation (SOSP 2023) · Llama-70B serving via this stack is standard.

Full Production System — Multi-Model Inference



KubeRay on Kubernetes · Autoscaling per deployment · max_restarts=5 on all GPU actors

Quantization + Ray — Complementary, Not Competing

Technique	Precision	VRAM (7B model)	Accuracy Loss	Use Case
FP32 (baseline)	32-bit	14 GB	None	Training / reference
BF16	16-bit	7 GB	Negligible	Most production serving
INT8 (GPTQ)	8-bit	3.5 GB	Low	Cost-sensitive serving
INT4 (AWQ)	4-bit	1.75 GB	Low–Medium	Edge / extreme density

Quantization and Ray operate at different layers and compose cleanly.

Quantize the model → serve it with Ray Serve. Quantization reduces per-model memory cost. Ray handles concurrency, routing, and scaling. They are not alternatives.

Three Things to Walk Away With

01

Inference is a systems problem

Training and inference are not symmetric. Inference is latency-sensitive, concurrent, and stateful. Treat it as systems design — not just model optimization.

02

Ray is the missing distributed layer

Its actor model maps naturally to stateful model servers. Ray Serve adds HTTP, autoscaling, batching, and deployment graphs on top. It is production-ready at scale.

03

The full stack is assembled now

KubeRay + Ray Serve + vLLM + quantization + continuous batching. The hard problems are solved. Your job is composing them correctly for your workload.

Further Reading



Ray Docs

docs.ray.io

Start: 'Ray Serve in Production' guide



ORCA Paper

OSDI 2022

Continuous batching — foundational LLM inference paper



vLLM

github.com/vllm-project/vllm

PagedAttention + Ray backend for LLM serving



PagedAttention

SOSP 2023

vLLM foundation — KV cache memory management



KubeRay

github.com/ray-project/kuberay

Ray on Kubernetes — production operator



Ray Paper

OSDI 2018

'Ray: A Distributed Framework for Emerging AI Apps'